PROBABILITY & STATISTICS LAB · ASSESSMENT 10 · PPT + R CODE

SilentHeat: presentation and R code

How R carries the whole study, from raw data to the final slide

Arhaansh Jhingan (25BCE0541) · Tvesha Rathi (25BCE2001) · Laksh Raja (25BCE0161)
SCOPE · VIT Vellore · Code: github.com/XArhaanshX/SilentHeat

DELIVERABLES

ITEM	LOCATION
Presentation (27 slides)	`presentation/SilentHeat.pptx`, with a PDF copy in `presentation/SilentHeat.pdf`
R code (~1,950 lines)	`run_all.R`, `install.R`, 12 scripts in `R/`, and the dashboard in `app/app.R`
Written report	`report/report.pdf`, generated from `report/report.Rmd`
Interactive dashboard	`Rscript -e 'shiny::runApp("app")'`

PRESENTATION OUTLINE

SLIDES	CONTENT
1–5	Title, problem statement, literature review, the gap, and our six research questions
6–9	Pipeline block diagram, data (ERA5, 10 cities, 1981–2025), and the R toolkit
10–15	Why dry-bulb misses humid heat; RQ1 blind spot (96.3% missed) with its evidence; seasons; distributions
16–19	RQ2 trends (×1.54 per decade), RQ3 extremes (GEV), RQ4 Markov spells, RQ5 logistic warning (AUC 0.92)
20–22	RQ6 baseline ML: set-up and novelty, model comparison (random forest AUC 0.95), hybrid alert and transfer to unseen cities
23–27	Shiny dashboard, conclusions and recommendations, limitations, references, thank you

Every chart on the slides is a ggplot2 figure produced by the R pipeline. Every number on the slides is read from the result tables R writes to `outputs/tables/`. Re-running the R code therefore updates the deck.

WHAT WE USED R FOR

STAGE	R SCRIPT	R PACKAGES	WHAT R DOES
Orchestration	`run_all.R`	base R	Runs all 11 steps in order with timing; rebuilds everything in about 1.5 minutes
Data collection	`01_fetch_data.R`	jsonlite	Calls the Open-Meteo API for ERA5 data, with retries and a local cache
Feature building	`02_build_features.R`	dplyr, tidyr	Harmonic-regression climate normals, the official IMD heatwave rule on every day, wet-bulb danger flags, heat index
Exploratory analysis	`03_eda.R`	ggplot2, ggridges	Descriptive statistics, Tmax–humidity correlation, wet-bulb regression, figures
Distributions	`04_distribution_fit.R`	fitdistrplus, MASS	MLE fits of Normal, Gamma, Weibull and Lognormal; Poisson vs negative binomial for counts
RQ1 Blind spot	`05_blind_spot.R`	base stats	Miss rate with Wilson, exact, block-bootstrap and Bayesian intervals; χ^2, McNemar, Cohen's κ
RQ2 Trends	`06_trends.R`	MASS	Mann–Kendall and Sen's slope, negative-binomial GLM, Wilcoxon, Welch, KS, permutation tests, ANOVA with Tukey
RQ3 Extremes	`07_extremes_gev.R`	own code, evd (check)	GEV maximum likelihood written from scratch, return levels, non-stationary GEV with likelihood-ratio test
RQ4 Spells	`08_markov_spells.R`	base R	Two-state Markov chain, geometric spell lengths, Monte Carlo simulation by era
RQ5 Warning	`09_early_warning.R`	pROC	Logistic regression, Brier skill score, ROC/AUC with DeLong CI, reliability diagram
RQ6 ML	`10_ml_models.R`	rpart, ranger	Decision tree and random forest at 1–3 day leads, hybrid IMD + ML alert, leave-one-city-out transfer
Report	`11_render_report.R`	knitr, markdown	Knits the report with live numbers and prints it to PDF
Dashboard	`app/app.R`	shiny, bslib	Blind spot, trends, extremes, and a risk calculator showing both verdicts

Several methods were written from first principles in `R/00_setup.R` instead of being called from a package: GEV density, likelihood and return levels; the Mann–Kendall test with tie correction and

Sen's slope; the Wilson confidence interval; Cohen's κ; Markov transition counts; Stull's wet-bulb formula; and the NWS heat index. We validated the GEV fit against `evd::fgev`, and the estimates agree to four decimal places.

CODE HIGHLIGHTS

run_all.R · lines 8–25

```r
Sys.setenv(SILENTHEAT_HOME = normalizePath("."))
source("R/00_setup.R")

steps <- c("01_fetch_data.R", "02_build_features.R", "03_eda.R",
           "04_distribution_fit.R", "05_blind_spot.R", "06_trends.R",
           "07_extremes_gev.R", "08_markov_spells.R", "09_early_warning.R", "10_ml_models.R",
           "11_render_report.R")

t_all <- Sys.time()
for (s in steps) {
  t0 <- Sys.time()
  message("\n========== ", s, " ==========")
  source(file.path("R", s), local = new.env(parent = globalenv()))
  message(sprintf("   finished in %.1f s", as.numeric(Sys.time() - t0, units = "secs")))
}
message(sprintf("\nAll steps done in %.1f min. Outputs in outputs/, report in report/.",
                as.numeric(Sys.time() - t_all, units = "mins")))
message("Launch the dashboard with:  Rscript -e 'shiny::runApp(\"app\")'")
```

Figure 1. **One command runs the whole study.** `run_all.R` sources each step in a fresh environment and times it.

R/02_build_features.R · lines 61–81

```r
d <- d |>
  mutate(
    imd_qualify = if_else(imd_category == "Plains",
                          (tmax >= 40 & departure >= 4.5) | tmax >= 45,
                          tmax >= 37 & departure >= 4.5),
    imd_severe_qualify = if_else(imd_category == "Plains",
                                 (tmax >= 40 & departure > 6.4) | tmax >= 47,
                                 tmax >= 37 & departure > 6.4)
  ) |>
  group_by(city) |>
  mutate(imd_heatwave = run_length_of(imd_qualify) >= 2,
         imd_severe   = imd_heatwave & imd_severe_qualify) |>
  ungroup()

# ---- 3. Humid-heat flags and the heat index ----------------------------------
d <- d |>
  mutate(
    danger    = twmax >= TW_DANGER,
    critical = twmax >= TW_CRITICAL,
    silent_danger = danger & !imd_heatwave,          # the blind spot
    heat_index = heat_index(tmax, rhmin),            # RH minimum ~ time of Tmax
```

Figure 2. **The official rule, applied to all 164,360 days.** The IMD plains and coastal criteria, the two-consecutive-day requirement, and the wet-bulb danger flag. `silent_danger` is the blind spot.

R/00_setup.R · lines 145–158

```r
wilson_ci <- function(x, n, conf = 0.95) {
  z <- qnorm(1 - (1 - conf) / 2)
  p <- x / n
  centre <- (p + z^2 / (2 * n)) / (1 + z^2 / n)
  half   <- z * sqrt(p * (1 - p) / n + z^2 / (4 * n^2)) / (1 + z^2 / n)
  data.frame(est = p, lo = centre - half, hi = centre + half)
}

cohens_kappa <- function(tab) {
  n  <- sum(tab)
  po <- sum(diag(tab)) / n
  pe <- sum(rowSums(tab) * colSums(tab)) / n^2
  (po - pe) / (1 - pe)
}
```

Figure 3. **Interval estimation and agreement, from scratch.** The Wilson score interval for the miss rate, and Cohen's κ between the IMD rule and humid danger.

R/00_setup.R · lines 192–216

```r
gev_nll <- function(par, z, t = NULL) {
  # par = (mu0, [mu1], log_sigma, xi); location linear in covariate t if given
  if (is.null(t)) { mu <- par[1]; sigma <- exp(par[2]); xi <- par[3] }
  else            { mu <- par[1] + par[2] * t; sigma <- exp(par[3]); xi <- par[4] }
  y <- 1 + xi * (z - mu) / sigma
  if (any(y <= 0) || sigma <= 0) return(1e10)
  if (abs(xi) < 1e-6) {
    s <- (z - mu) / sigma
    return(length(z) * log(sigma) + sum(s) + sum(exp(-s)))
  }
  length(z) * log(sigma) + (1 + 1 / xi) * sum(log(y)) + sum(y^(-1 / xi))
}

fit_gev <- function(z, t = NULL) {
  start <- c(mean(z) - 0.45 * sd(z), log(0.78 * sd(z)), -0.1)
  if (!is.null(t)) start <- c(start[1], 0, start[2:3])
  best <- NULL
  for (xi0 in c(-0.3, -0.1, 0.1)) {
    s <- start; s[length(s)] <- xi0
    f <- optim(s, gev_nll, z = z, t = t, method = "Nelder-Mead",
               control = list(maxit = 5000, reltol = 1e-12))
    f <- optim(f$par, gev_nll, z = z, t = t, method = "BFGS",
               control = list(maxit = 1000, reltol = 1e-12))
    if (is.null(best) || f$value < best$value) best <- f
  }
```

Figure 4. **Extreme-value theory by maximum likelihood.** The GEV negative log-likelihood (with an optional linear trend in location), minimised with Nelder–Mead and then BFGS from several starting points.

R/08_markov_spells.R · lines 31–37

```r
simulate_season <- function(n_days, p01, p11) {
  pi1 <- p01 / (p01 + 1 - p11)
  s <- logical(n_days); s[1] <- runif(1) < pi1
  u <- runif(n_days)
  for (i in 2:n_days) s[i] <- u[i] < if (s[i - 1]) p11 else p01
  s
}
```

Figure 5. **Markov chain Monte Carlo of a season.** Each day's state depends only on yesterday's, through the transition probabilities p_{01} and p_{11}.

R/10_ml_models.R · lines 25–39

```r
  arrange(city, date) |>
  group_by(city) |>
  mutate(tw_lag1 = lag(twmax), tw_lag2 = lag(twmax, 2),
         tw_change = twmax - tw_lag1,
         tw_roll3 = (twmax + tw_lag1 + tw_lag2) / 3,
         tw_max7 = pmax(twmax, lag(twmax), lag(twmax, 2), lag(twmax, 3),
                        lag(twmax, 4), lag(twmax, 5), lag(twmax, 6)),
         rh_change = rhmin - lag(rhmin),
         # physiology-informed: headroom to the danger limit and its closing speed
         headroom = TW_DANGER - twmax,
         closing = (tw_roll3 - lag(tw_roll3, 3)) / 3,
         days_to_limit = pmin(pmax(headroom, 0) / pmax(closing, 0.05), 30),
         danger_days7 = danger + lag(danger) + lag(danger, 2) + lag(danger, 3) +
           lag(danger, 4) + lag(danger, 5) + lag(danger, 6)) |>
  ungroup() |>
```

Figure 6. **Physiology-informed features (novelty).** Headroom to the 29 °C limit, its closing speed, and estimated days-to-limit, alongside lagged and rolling wet-bulb history.

R/10_ml_models.R · lines 62–85

```r
fit_predict <- function(train, test, use_city = TRUE) {
  x <- c(feats, if (use_city) "city" else geo)
  f <- reformulate(x, "yf")
  f_lr <- reformulate(c(setdiff(x, "city"), if (use_city) "city"), "y")
  # climatology: training-period danger frequency by city (or coast type) and month
  key <- if (use_city) "city" else "coastal"
  clim <- train |> group_by(across(all_of(c(key, "month")))) |>
    summarise(p_clim = mean(y), .groups = "drop")
  lr <- suppressWarnings(glm(f_lr, binomial, train, control = glm.control(maxit = 100)))
  # probability tree: regression on 0/1 (Brier loss); a class tree on 2%
  # positives never beats the all-"no" root and prunes to nothing
  set.seed(2026)                        # rpart's cross-validation folds are random
  tr <- rpart(reformulate(x, "y"), train, method = "anova",
              control = rpart.control(cp = 5e-4, minbucket = 40, xval = 5))
  tr <- prune(tr, cp = tr$cptable[which.min(tr$cptable[, "xerror"]), "CP"])
  rf <- ranger(f, train, num.trees = 400, min.node.size = 25, probability = TRUE,
               importance = "permutation", num.threads = 8, seed = 2026)
  list(models = list(lr = lr, tree = tr, rf = rf),
       p = data.frame(Climatology = coalesce(left_join(test, clim, by = c(key, "month"))$p_clim, 0),
                      Logistic = predict(lr, test, type = "response"),
                      `Decision tree` = predict(tr, test),
                      `Random forest` = predict(rf, test)$predictions[, "yes"],
                      check.names = FALSE))
}
```

Figure 7. **The baseline model zoo.** Climatology, logistic regression, a pruned decision tree (rpart) and a 400-tree random forest (ranger), all fitted on the same data and scored on unseen years.

app/app.R · lines 65–73

```r
  nav_panel("Risk calculator",
    layout_columns(col_widths = c(4, 8),
      card(card_header("Today's weather"),
           dateInput("date", "Date", value = as.Date("2026-05-20")),
           numericInput("tmax", "Max temperature (°C)", 36, 20, 50, 0.1),
           numericInput("rhmin", "Min relative humidity (%)", 60, 1, 100, 1),
           numericInput("tmin", "Min temperature (°C)", 29, 10, 40, 0.1),
           numericInput("twy", "Yesterday's max wet-bulb (°C)", 28.5, 10, 35, 0.1)),
      card(card_header("Two verdicts for the same day"), uiOutput("verdict")))
```

Figure 8. **The Shiny risk calculator.** The user enters today's weather and the app shows the IMD verdict next to the physiological and model verdicts.

HOW TO RUN

```
Rscript install.R                   # install the 17 R packages
Rscript run_all.R                   # data → features → 6 analyses → ML → figures → report
Rscript -e 'shiny::runApp("app")'   # interactive dashboard
```